Names: Christian Ezekiel L. Carvajal & John Miko P. Sarsalijo	**Subject: BSCS 3112 - AI Lab (Lesson 7 Final Project)**
Course/Year: BSCS - DS 3rd year	**Date: October 22, 2026**

FINAL PROJECT MILESTONE 1: AI VIRTUAL WAITER FOR PERPETUAL BISTRO

1. Executive Summary & Business Scenario

Perpetual Bistro, the primary on-campus dining facility at the University of Perpetual Help System DALTA (UPHSD), requires an automated table-side digital ordering assistant. During peak meal rushes, counter ordering encounters severe line congestion, communication bottlenecks regarding dietary concerns, and manual entry errors. To modernize service, our engineering pair developed an autonomous, multi-modal conversational ordering agent.

To guarantee complete privacy, eliminate external cloud API subscriptions, and ensure continuous operation during campus network interruptions, the application runs entirely locally on edge hardware. Milestone 1 constructs the foundational backend: combining a decoupled speech I/O pipeline (speech_engine.py), an authoritative Finite State Machine (fsm_controller.py), and an SQLite relational tool engine (db_tools.py, restaurant_db.sql, and init_db.py) powered by local Ollama execution with qwen2.5.

2. PEAS Framework Application: Deconstruction of the BistroBot Agent

To systematically establish the operating envelope of the Perpetual Bistro Virtual Waiter, we apply the PEAS (Performance Measure, Environment, Actuators, Sensors) framework:

- Performance Measure: Evaluates transaction accuracy and conversational reliability. Metrics include: (1) 100% order accuracy with 0 hallucinated dishes or fabricated prices; (2) Zero negative inventory balances enforced by ACID transaction rollbacks; (3) Conversational response latency under 1.5 seconds on local Ollama hardware; (4) 100% allergen detection rate for flagged dietary restrictions; and (5) Itemized receipt transparency upon checkout.
- Environment: The operating domain encompasses table-side digital terminals across dining tables in Perpetual Bistro. The environment is partially observable (customer desires unfold dynamically turn-by-turn), stochastic (user phrasing varies between English, Taglish, and sudden cancellations), sequential (each turn directly alters cart state), and dynamic (kitchen stock depletes in real time).
- Actuators: Mechanisms through which the agent manipulates its environment. These include: (1) SQLite database DML operations (add_item_to_order, update_item_quantity, confirm_order_and_deduct_stock); (2) Spoken audio synthesis via Edge-TTS and offline pyttsx3; (3) Dynamic state transitions inside fsm_controller.py; and (4) Digital kitchen ticket issuance (#PB-table-uuid).
- Sensors: Channels through which the agent receives environmental data. These include: (1) SpeechRecognition microphone audio capture; (2) Text input streams via CLI or HTTP; (3) Real-time SQLite catalog and stock level query readers; and (4) Internal Finite State Machine state variables.

3. Conversational Finite State Machine (FSM) Architecture

The conversational workflow is coordinated by an unbypassable Finite State Machine (FSM) implemented in fsm_controller.py. Rather than permitting unconstrained language model generations, dialogue progresses through 5 discrete states synchronized with the table_sessions table in SQLite:

- STATE_GREETING: Initializes session context, welcomes the guest to Perpetual Bistro, captures the physical table number, and prompts for initial dining requests.
- STATE_MENU_BROWSING: Engages when customers inquire about available dishes, category listings (Burgers, Pastas, Sides, Beverages, Desserts, Combos), prices, or allergen profiles.

- STATE_TAKING_ORDER: Triggers when the guest selects dishes. The agent invokes add_item_to_order, binding custom preparation instructions ('No onions, extra cheese').
- STATE_REVIEW: Activates when the customer requests their tab or initiates checkout. The agent presents an itemized breakdown with subtotal, 12% VAT, and grand total.
- STATE_CONFIRMED: The terminal state reached upon customer confirmation. The engine executes atomic inventory decrements, prints an immutable digital ticket, dispatches the order to the kitchen queue, and locks the cart against further edits.

Bidirectional transition guards prevent illegal operations. A customer in STATE_REVIEW can add further beverages, returning the session to STATE_TAKING_ORDER. Conversely, attempting to skip directly from STATE_GREETING to STATE_CONFIRMED with an empty cart is blocked by state validation.

4. Relational Database Modeling & Inventory Integrity

Persistence is anchored in SQLite (restaurant.db), initialized via init_db.py executing init_restaurant_db.sql. The schema structures operational data across four normalized tables and an active view:

- menu_items: Catalog of 21 authentic campus dishes across 6 categories (Burgers, Pastas, Sides, Beverages, Desserts, Combos), recording prices, stock quantities (stock_qty), and allergen tags.
- table_sessions: Tracks table numbers, session creation timestamps, and active FSM states.
- order_line_items & cart_items view: Stores active cart items, quantities, special instructions, and line totals linked via foreign keys.
- kitchen_tickets: Manages back-of-house kitchen fulfillment queues (PREPARING -> READY -> SERVED) and payment audit trails.

To test inventory safeguards, the database includes an intentionally depleted dish: 'Crispy Buffalo Chicken Tenders' with stock_qty set to 0. When ordered, database guards

intercept the request, reject insertion, query available dishes in the category, and instruct the agent to propose alternatives.

5. Decoupled Voice I/O Pipeline Architecture

In compliance with Phase 3 Milestone 1 requirements, speech processing is decoupled from the core agent engine inside speech_engine.py. The SpeechEngine class exposes abstract input/output adapters:

- Speech-to-Text (STT): Captures microphone audio using SpeechRecognition with automatic ambient noise calibration and silence thresholding, converting spoken customer requests into text strings.
- Text-to-Speech (TTS): Synthesizes agent response strings into vocal audio using Edge-TTS neural voices (en-PH-JamesNeural / en-PH-RosaNeural) with offline pyttsx3 fallback.
- Phonetic Taglish Normalization: The normalize_taglish_text method converts currency formatting (₱150 -> 150 pesos), expands campus acronyms (UPHSD -> Perpetual Help System DALTA), and strips markdown syntax, ensuring natural voice delivery.

6. Search Strategy Analysis: Uninformed vs. Informed Dialogue Traversal

Navigating dialogue state spaces and menu graph hierarchies requires comparing uninformed and informed search strategies across exact computational complexities:

- Uninformed Search: (1) Breadth-First Search (BFS) explores the dialogue state space level-by-level using a FIFO queue. Time complexity is O(b^d) and space complexity is O(b^d), where b is branching factor (average 3 intents per turn) and d is state depth (d=4 from GREETING to CONFIRMED). BFS guarantees finding the shallowest path to order completion. (2) Depth-First Search (DFS) probes conversational branches to maximum depth using a LIFO stack. Time complexity is O(b^m) and space complexity is O(b*m), where m is maximum depth. While space-efficient, DFS risks getting trapped in recursive ingredient sub-inquiries without cycle detection. (3) Uniform-Cost Search expands paths

based on minimal cumulative cost g(n) (e.g., preparation time or billing cost), taking time and space O(b^(1 + floor(C*/epsilon))).

- Informed Search: (1) Greedy Best-First Search guides state progression via evaluation function f(n) = h(n). While fast, it is neither complete nor optimal. (2) A* Search combines path cost with heuristic estimation: f(n) = g(n) + h(n). A* is proven complete and optimal when the heuristic h(n) is admissible and consistent, minimizing dialogue turns while avoiding out-of-stock and allergen hazards.

7. Task Scheduling Arrays: BFS and DFS Multi-Turn Implementation

To manage multi-turn intent queues and compound orders, the system implements dual task scheduling arrays using standard library data structures:

- BFS Task Scheduling Array (FIFO Queue via collections.deque): Used for horizontal category exploration and batch constraint resolution. When a customer asks 'What pastas do you have and are any nut-free?', the scheduler enqueues 'query_category' and 'check_nut_allergen'. Tasks execute in sequence via popleft(), guaranteeing that broad menu inquiries resolve before item additions proceed.
- DFS Task Scheduling Array (LIFO Stack via Python List): Used for hierarchical item customization. When a customer orders a complex combo, customization steps (Meal Selection -> Side Selection -> Beverage Option -> Custom Instructions) push onto the stack via append(). The agent resolves the most recently nested customization via pop() before finalizing the parent order line.

8. Heuristic Function Design: Admissible & Consistent Manhattan Distance

When a requested dish is depleted (e.g., Crispy Buffalo Chicken Tenders with stock=0), the agent must recommend the closest culinary substitute without human intervention. We designed an admissible and consistent heuristic function based on Manhattan Distance in normalized menu feature space:

Let an item be represented by vector v = (category_id, price, prep_time). The heuristic distance between candidate item n and target item g is formulated as: h_M(n, g) = w_cat * |cat_n - cat_g| + w_price * (|price_n - price_g| / price_max) + w_prep *

(|prep_n - prep_g| / prep_max), with weights w_cat = 2.0, w_price = 1.0, and w_prep = 0.5.

- Admissibility Proof: An admissible heuristic satisfies h(n) <= h*(n), where h*(n) is the true optimal substitution cost. Because Manhattan distance calculates orthogonal feature differences and assigns zero cost to identical items, h_M(n, g) never overestimates the true culinary penalty. This confirms that h_M is strictly admissible.
- Consistency (Monotonicity) Proof: A heuristic is consistent if for every node n and successor n' generated by action a with step cost c(n, a, n'), the condition h(n) <= c(n, a, n') + h(n') holds. Because Manhattan distance satisfies the triangle inequality across all normalized feature dimensions, monotonicity is strictly preserved. In automated tests, h_M avoided out-of-stock items and identified Perpetual Classic Burger (h_M = 0.0833) and Bacon Mushroom Melt Burger (h_M = 0.2583) as optimal substitutes.

9. Programmatic Guardrails & Defense Stress Testing

The architecture incorporates programmatic defenses verified by test_edge_cases.py to satisfy the instructor's live defense requirements:

- Out-of-Stock Protection: db_tools.add_item_to_order queries stock_qty prior to insertion. If stock is 0, the commit is blocked, and available dishes in the category are returned.
- Post-Confirmation Lockout: Once an order reaches STATE_CONFIRMED, all modification tools (add, update, remove) are locked, preventing alterations after kitchen dispatch.
- Empty Order Checkout Guard: Transitioning to STATE_REVIEW or STATE_CONFIRMED is blocked if active cart items equal 0.
- Out-of-Domain Redirection: System prompt boundaries politely deflect non-dining inquiries and re-anchor the diner to menu offerings.

10. Tandem Pair Programming Reflection & Milestone 1 Allocation

In accordance with the Extreme Programming (XP) Pair Programming paradigm, our team alternated Driver and Navigator responsibilities during Milestone 1:

- Milestone 1 Driver (Christian Ezekiel L. Carvajal): Authored relational SQLite schema (restaurant_db.sql, init_restaurant_db.sql), constructed database CRUD wrappers (db_tools.py), implemented the decoupled speech pipeline (speech_engine.py), and built the CLI integration harness (cli_waiter.py).
- Milestone 1 Navigator (John Miko P. Sarsalijo): Architected FSM state transitions (fsm_controller.py), audited Ollama tool schemas for Qwen 2.5, verified out-of-stock rollback logic, and validated acoustic Taglish phonetic pronunciation rules.

11. Verification Evidence & Milestone 1 Execution

Milestone 1 was verified across three automated test suites:

- Database Initialization (init_db.py): Successfully created restaurant.db with 21 menu items across 6 categories and verified stock depletion constraints.
- Edge-Case Stress Harness (test_edge_cases.py): Passed all 4 tests with 100% accuracy, confirming out-of-stock interception, tab calculation, stock deduction, and post-confirmation lockout.
- CLI Integration Loop (cli_waiter.py): Demonstrated that voice and text inputs drive FSM state transitions from STATE_GREETING to STATE_CONFIRMED, updating SQLite without GUI dependencies.

12. Conclusion & Milestone 2 Transition Roadmap

Lesson 7 Milestone 1 completes the foundational multi-modal ordering engine. By coupling a decoupled voice pipeline with an FSM controller, relational database tools, and admissible heuristic substitution, the system operates with zero cloud reliance. Milestone 1 establishes the verified foundation for Milestone 2, where the web UI dashboard (ui_dashboard/) and end-to-end video walkthrough will be finalized.